

鹰眼长空—开发文档

Contents

1 鹰眼长空—开发文档	2
1.1 1. 项目概述	2
1.2 2. 环境搭建	2
1.2.1 2.1 依赖安装	2
1.2.2 2.2 配置	2
1.2.3 2.3 启动	3
1.2.4 2.4 测试数据	3
1.3 3. 项目架构	3
1.3.1 3.1 目录总览	3
1.3.2 3.2 架构分层	5
1.3.3 3.3 数据流	5
1.4 4. 核心概念	5
1.4.1 4.1 全局状态 (state.py)	5
1.4.2 4.2 备份快照结构	6
1.4.3 4.3 .dat 文件格式	6
1.5 5. API 参考 (13 个端点)	6
1.5.1 数据管理	6
1.5.2 预测 & AI	6
1.5.3 报告	7
1.5.4 备份管理	7
1.5.5 分析	7
1.5.6 统一响应格式	7
1.6 6. 前端架构	7
1.6.1 6.1 页面结构	7
1.6.2 6.2 CSS 设计令牌	7
1.6.3 6.3 Toast 通知	8
1.6.4 6.4 添加新页面	8
1.7 7. 添加新功能指南	8
1.7.1 7.1 添加新 API 端点	8
1.7.2 7.2 添加新算法	8
1.7.3 7.3 添加新的检测平台	9
1.8 8. 调试技巧	9
1.8.1 8.1 查看注册的路由	9
1.8.2 8.2 调试数据加载	9
1.8.3 8.3 查看备份内容	9
1.8.4 8.4 Windows 终端乱码	9

1.9 9. 常见问题 9

1.10 10. 代码规范 10

1 鹰眼长空—开发文档

面向新开发者的上手指南

1.1 1. 项目概述

鹰眼长空是一个桌面端无人机轨迹监测与预测系统，融合可见光、红外、雷达三种传感平台的轨迹数据，提供 3D 可视化、轨迹预测、AI 策略生成和多维运动学分析。

技术栈：

层	技术
后端框架	Python Flask（蓝图路由）
桌面容器	pywebview（Windows 原生窗口）
3D 渲染	Three.js（CDN）+ 自定义 WebGL
图表	ECharts 5（CDN）
AI 服务	硅基流动 API（Qwen2.5-7B-Instruct）
前端样式	纯 CSS（Apple HIG 深色主题）
数据格式	自定义.dat（每行 x y z t）

1.2 2. 环境搭建

1.2.1 2.1 依赖安装

```
# Python 3.10+
pip install flask pywebview requests
```

1.2.2 2.2 配置

编辑根目录 config.json：

```
{
  "siliconflow": {
    "api_key": " 你的 API 密钥",
    "url": "https://api.siliconflow.cn/v1/chat/completions",
    "model": "Qwen/Qwen2.5-7B-Instruct"
  },
  "detection_methods": {
    "visible": {"name": " 可见光", "color": "#FF3B30", "visible": true},
    "infrared": {"name": " 红外", "color": "#34C759", "visible": true},
    "radar": {"name": " 雷达", "color": "#FFCC00", "visible": true}
  }
}
```

```

    },
    "prediction_settings": {
        "min_points": 1,
        "max_points": 20,
        "default_points": 6,
        "time_step": 0.5
    }
}

```

1.2.3 2.3 启动

桌面窗口 (*Windows*)

```
python main.py
```

纯 *HTTP* 服务 (调试推荐)

```
python main.py recon --headless
```

仅轨迹识别

```
python main.py recog
```

同时启动两个模块

```
python main.py all --headless
```

独立模块入口

```
python -m trajectory_reconstruction --headless
```

```
python -m trajectory_recognition
```

1.2.4 2.4 测试数据

项目自带 3 组虚构轨迹数据, 位于 `data/fact/`:

文件	平台	点数	特征
fact1.dat	可见光	60	Z 字形起伏飞行
fact2.dat	红外	60	盘旋上升飞行
fact3.dat	雷达	60	S 形曲线加速飞行

1.3 3. 项目架构

1.3.1 3.1 目录总览

```

drone/
├── main.py                # 统一入口 (支持 recon/recog/all 子命令)
├── config.json            # 运行时配置 (不入库)
└── trajectory_reconstruction/  # 轨迹重建与分析 (核心模块)

```

```

├── app.py # Flask 应用工厂
├── __main__.py # python -m 独立入口
├── core/ # 领域逻辑层 (纯函数, 无 Flask 依赖)
│   ├── state.py # 全局内存缓存 + API 配置
│   ├── config/config_manager.py # 配置文件读写/校验/默认值
│   ├── io/data_loader.py # .dat 文件 I/O
│   ├── prediction/prediction.py # 线性外推预测算法
│   └── ai/ai_service.py # 硅基流动 API 调用
├── services/ # 业务逻辑层 (编排 core 模块)
│   ├── data_service.py # 数据加载/刷新/清理
│   ├── predict_service.py # 参数校验 + 预测编排
│   └── backup_service.py # 快照备份/列表/恢复/删除
├── views/ # HTTP 接口层 (Flask 蓝图, 薄层)
│   ├── __init__.py # 蓝图注册 + 错误处理
│   ├── main.py # 5 个页面路由
│   ├── api.py # 数据 & 备份 API
│   ├── api_predict.py # 预测 & AI API
│   ├── api_report.py # 报告保存 API
│   └── analysis.py # 运动学分析数据 API
├── frontend/ # 前端资源
│   ├── pages/ # Jinja2 模板 (5 页面 + base)
│   │   └── static/
│   │       ├── css/main.css # 全局样式 (Apple HIG 深色主题)
│   │       └── js/
│   │           ├── common/toast.js # Toast 通知组件
│   │           └── pages/ # 每页独立 JS 模块
├── trajectory_recognition/ # 轨迹识别模块 (框架, 待实现)
│   ├── app.py # Flask 应用工厂 (端口 5001)
│   ├── __main__.py
│   ├── features/ # 特征提取 (TODO)
│   ├── models/ # 模型定义 (TODO)
│   └── classifier/ # 分类器 (TODO)
├── data/ # 共享运行时数据
│   ├── fact/ # 实际轨迹 (.dat)
│   ├── predict/ # 预测结果
│   └── backup/ # 快照备份
├── reports/ # AI 策略报告
├── templates/ # 配置模板
└── docs/ # 文档

```

1.3.2 3.2 架构分层

frontend/	(HTML/CSS/JS)		← 浏览器渲染
views/	(Flask 蓝图)		← HTTP 解析, JSON 响应
services/	(业务编排)		← 参数校验, 流程控制
core/	(领域逻辑)		← 算法、I/O、AI、配置

单向依赖: views → services → core。core 不依赖 Flask, services 不处理 HTTP。

1.3.3 3.3 数据流

```
data/fact/*.dat          ← 原始轨迹文件
  ↓ load_default_data()
detection_methods (state.py) ← 全局内存缓存
  ↓ generate_prediction()
data/predict/*.dat       ← 预测输出
  ↓
trajectory_recognition   ← 未来: 读取共享数据进行识别
```

1.4 4. 核心概念

1.4.1 4.1 全局状态 (state.py)

```
# 模块级字典, 加载后常驻内存
detection_methods: dict[str, dict] = {
    "visible": {
        "name": " 可见光",
        "color": "#FF3B30",
        "visible": True,
        "points": [[x,y,z], ... ],      # 轨迹坐标
        "timestamps": [t, ... ],       # 时间戳
    },
    "infrared": { ... },
    "radar": { ... },
    # "self": { ... } ← 用户上传的自选平台, 动态创建
}
```

生命周期: 1. 启动时 `init_from_config()` 读取 `config.json` 填充元信息 2. `initialize_data()` 从 `data/fact/` 加载轨迹点 3. 运行中通过 API 增删改 4. 元信息变更时 `save_metadata()` 写回 `config.json`

1.4.2 4.2 备份快照结构

```
data/backup/20260629_143052_auto/
├── manifest.json          # {created, label, methods, files}
├── fact/                  # 复制自 data/fact/
├── predict/               # 复制自 data/predict/
└── memory/               # 内存 dump (visible.dat, ...)
```

label 为 auto (clear_all_data 时自动创建) 或 manual (用户手动创建)。

1.4.3 4.3 .dat 文件格式

```
x y z t
3.5 12.0 7.2 0.0
3.8 12.3 7.5 0.5
...
```

字段	含义	单位
x	水平位置 (东西)	米
y	高度	米
z	水平位置 (南北)	米
t	时间戳	秒

1.5 5. API 参考 (13 个端点)

所有接口基于 api_bp 蓝图, 基础路径 `http://127.0.0.1:5000`。

1.5.1 数据管理

方法	路径	说明
POST	/api/refresh_data	重新加载 data/fact/
POST	/api/load_data	上传.dat (multipart, method_id=self)
POST	/api/clear_all_data	自动备份后清空数据

1.5.2 预测 & AI

方法	路径	说明
POST	/api/predict	单平台预测 {method_id, points, timestamps?, num_points?, time_step?}
POST	/api/predict_all	全平台预测 {num_points?, time_step?}
POST	/api/ai_suggestion	获取 AI 策略 {methods_data}

1.5.3 报告

方法	路径	说明
POST	/api/save_report	保存报告到 reports/ {content, platforms?}

1.5.4 备份管理

方法	路径	说明
GET	/api/list_backups	列出快照 (按时间倒序)
POST	/api/restore_backup	恢复指定快照 {backup_file}
POST	/api/restore_all_backups	恢复最新快照
POST	/api/backup/create	手动创建快照
POST	/api/backup/delete	删除快照 {backup_name}

1.5.5 分析

方法	路径	说明
GET	/analysis/data	返回各平台速度/加速度/曲率/高度

1.5.6 统一响应格式

```
{"success": true, ... }  
{"success": false, "error": " 错误描述"}
```

1.6 6. 前端架构

1.6.1 6.1 页面结构

5 个页面均继承 base.html (导航栏 + toast 容器), 各自独立:

页面	文件	核心 JS	渲染引擎
总览	index.html	dashboard.js	Three.js 3D
预测	predict.html	predict.js	Three.js 3D
分析	analysis.html	内联脚本	ECharts
AI	ai.html	ai.js	DOM
数据	data.html	data.js	DOM + 模态框

1.6.2 6.2 CSS 设计令牌

所有颜色/圆角/阴影/过渡定义在 main.css 的 :root 中:

```
--blue: #0A84FF;          /* 品牌蓝 */
--bg-root: #0D0D0F;        /* 最深背景 */
--bg-card: #2C2C2E;        /* 卡片表面 */
--radius-2xl: 24px;        /* 卡片圆角 */
--radius-pill: 980px;      /* 按钮圆角 */
--shadow-lg: 0 8px 24px rgba(0,0,0,0.4);
--spring: cubic-bezier(0.175, 0.885, 0.32, 1.275);
```

1.6.3 6.3 Toast 通知

```
import { toast } from '../common/toast.js';
toast.success('操作成功');
toast.error('操作失败');
toast.warning('警告信息');
```

1.6.4 6.4 添加新页面

1. 在 frontend/pages/ 创建 newpage.html, 继承 base.html
2. 在 frontend/static/js/pages/ 创建 newpage.js
3. 在 views/main.py 添加路由:

```
@main_bp.route('/newpage')
def newpage_page():
    return render_template('newpage.html', **page_context('newpage'))
```

4. 在 base.html 导航栏添加链接

1.7 7. 添加新功能指南

1.7.1 7.1 添加新 API 端点

在 views/api.py 中添加:

```
@api_bp.route('/api/my_feature', methods=['POST'])
def my_feature():
    data = request.get_json(silent=True) or {}
    # 参数校验
    if not data.get('required_param'):
        return jsonify({'success': False, 'error': '缺少参数'}), 400
    # 调用 service
    result = my_service.do_something(data)
    return jsonify({'success': True, 'result': result})
```

1.7.2 7.2 添加新算法

在 core/ 下新建模块, 保持纯函数风格:


```
# core/my_algo/algorithm.py
def compute(data: list, params: dict) -> dict:
    """ 纯函数: 输入 → 计算 → 输出, 无副作用 """
    ...
    return result
```

然后在 services/ 层编排调用。

1.7.3 7.3 添加新的检测平台

1. 在 config.json 的 detection_methods 中添加新条目
2. 在 data/fact/ 放置对应的.dat 文件
3. 在 data_loader.py 的 load_default_data() 中添加映射
4. 重启即可

1.8 8. 调试技巧

1.8.1 8.1 查看注册的路由

```
python -c "
from trajectory_reconstruction.app import create_app
for r in sorted(create_app().url_map.iter_rules(), key=lambda x: x.rule):
    print(f'{r.rule:40s} {sorted(r.methods)}')
"
```

1.8.2 8.2 调试数据加载

```
from trajectory_reconstruction.core.state import detection_methods
for mid, data in detection_methods.items():
    print(f'{mid}: {len(data["points"])} points')
```

1.8.3 8.3 查看备份内容

```
ls -R data/backup/20260629_143052_auto/
cat data/backup/20260629_143052_auto/manifest.json
```

1.8.4 8.4 Windows 终端乱码

Git Bash 或 PowerShell 中可能出现 GBK 编码问题。在 VS Code 终端中设置 "terminal.integrated.defaultProfile.windows": "Git Bash" 可解决。

1.9 9. 常见问题

Q: data/fact/ 中文件被清空了怎么办?

```
git restore data/fact/
```

Q: 如何完全重置环境?

```
# 删除运行时数据
rm -rf data/backup/* data/predict/* reports/*
git restore data/fact/
```

Q: 如何更换预测算法?

修改 `core/prediction/prediction.py` 的 `generate_prediction()` 函数。保持函数签名不变即可无缝替换。

Q: 如何接入其他大模型?

修改 `core/ai/ai_service.py` 的 API URL、模型名和请求格式。`views/api_predict.py` 中调用时传入对应的 `SF_API_KEY` 等参数。

1.10 10. 代码规范

- **Python:** 函数签名使用类型注解 (`list[list[float]]`、`dict[str, Any]`)
- **JavaScript:** ES6 模块, `async/await`, 避免全局变量
- **CSS:** 所有颜色/尺寸使用 CSS 变量, 不硬编码
- **提交信息:** 中文描述, 简洁明了
- **导入顺序:** 标准库 → 第三方 → 项目模块 (空行分隔)
- **分层原则:** `core` 不导入 `Flask/services`; `services` 不导入 `views`; `views` 可导入任意层